

实验报告二

赵莘萌

9.7

目录

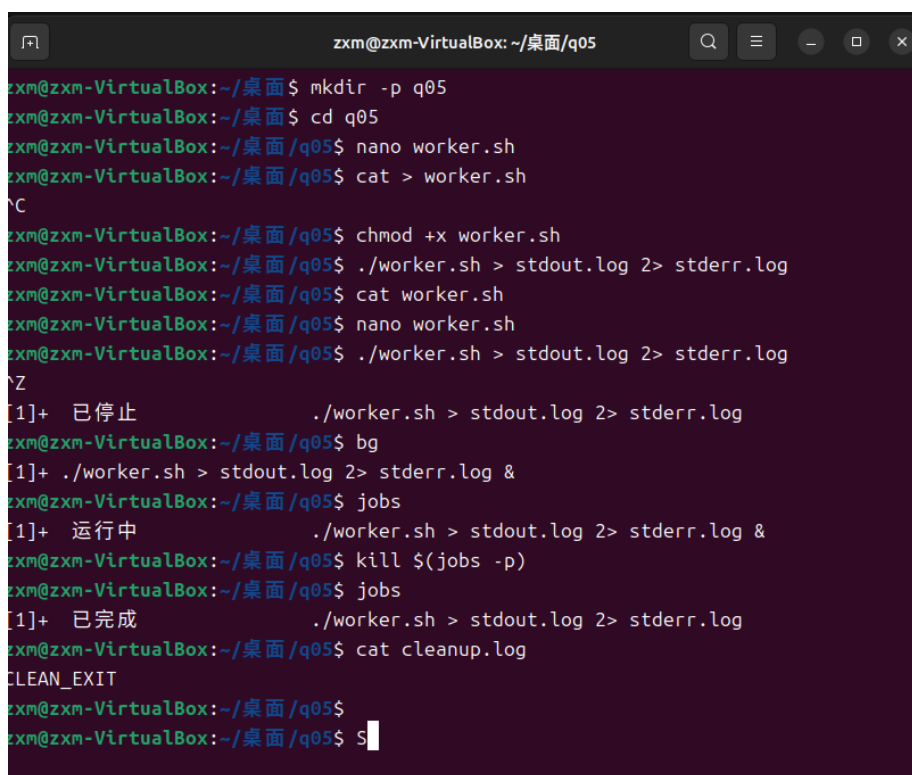
1	课上练习	1
1.1	第一题	1
1.2	第二题	2
1.3	第三题	6
1.4	第四题	9
2	课后习题	12
2.1	1. 你可能见过像 <code>cmd -flag --notafg</code> 这样的命令。这里的 <code>-</code> 是一个特殊参数，它告诉程序后面不要再继续解析选项 (flag) 了。也就是说， <code>-</code> 后面的所有内容都会被当作位置参数 (positional argument)。这有什么用？试着运行 <code>touch -myfile</code> ，然后在不使用 <code>-</code> 的情况下把它删掉。	12
2.2	2. 阅读 <code>man ls</code> ，写出一个 <code>ls</code> 命令，让它按下面的方式列出文件：包含所有文件，包括隐藏文件；文件大小以易读格式显示（例如 454M，而不是 454279954）；按最近修改时间排序；输出带颜色	13
2.3	3. 进程替换 <code><(command)</code> 可以让你把一个命令的输出当成文件来用。试着配合 <code>diff</code> 和进程替换，比较 <code>printenv</code> 与 <code>export</code> 的输出。它们为什么不一样？（提示：可以试试 <code>diff <(printenv sort) <(export sort)</code> ）	14

2.4	4. 写两个 bash 函数 marco 和 polo, 行为如下: 每次执行 marco 时, 都要以某种方式保存当前工作目录; 之后无论你切到哪个目录, 只要执行 polo, 它都应该把你 cd 回执行 marco 时所在的目录。为了方便调试, 你可以把代码写进 marco.sh, 然后通过执行 source marco.sh 把这些定义重新加载到当前 shell。	17
2.5	5. 假设你有一个很少失败的命令。为了调试它, 你需要把它的输出保存下来, 但等到一次失败运行可能会很耗时。写一个 bash 脚本, 不断运行下面这个脚本直到它失败为止, 并把标准输出和标准错误分别保存到文件里, 最后把结果打印出来。如果你还能顺便报告它运行了多少次才失败, 就加分。 .	18
2.6	6. 在终端里启动一个 sleep 100000 任务, 用 Ctrl-z 把它挂起, 再用 bg 让它继续在后台运行。然后使用 pgrep 找到它的 pid, 再用 pkill 把它杀掉, 整个过程都不要手动输入这个 pid。(提示: 试试 -af 参数)	19
2.7	7. 进入 /.ssh/, 检查你是否已经有一对 SSH 密钥。如果没有, 就用 ssh-keygen -a 100 -t ed25519 生成一对。建议给密钥设置密码, 并配合使用 ssh-agent。	20
2.8	8. 编辑.ssh/config, 写入 ssh 配置片段。进入 /.ssh 目录, nvim config 新建 / 编辑 config 文件; 填入配置, 替换 User、Host-Name 两处占位符; chmod 600 /.ssh/config 设置文件权限; 使用 ssh vm 测试连接, LocalForward 实现本地端口 9999 转发到远程 8888。	21
2.9	9. 使用 ssh-copy-id vm 把你的 SSH 公钥复制到服务器上。 .	23
2.10	10. 在虚拟机里执行 python -m http.server 8888 启动一个 Web 服务器。然后在你自己的机器上访问 http://localhost:9999, 确认自己能访问虚拟机里的这个服务。	23
3	解题感悟	24

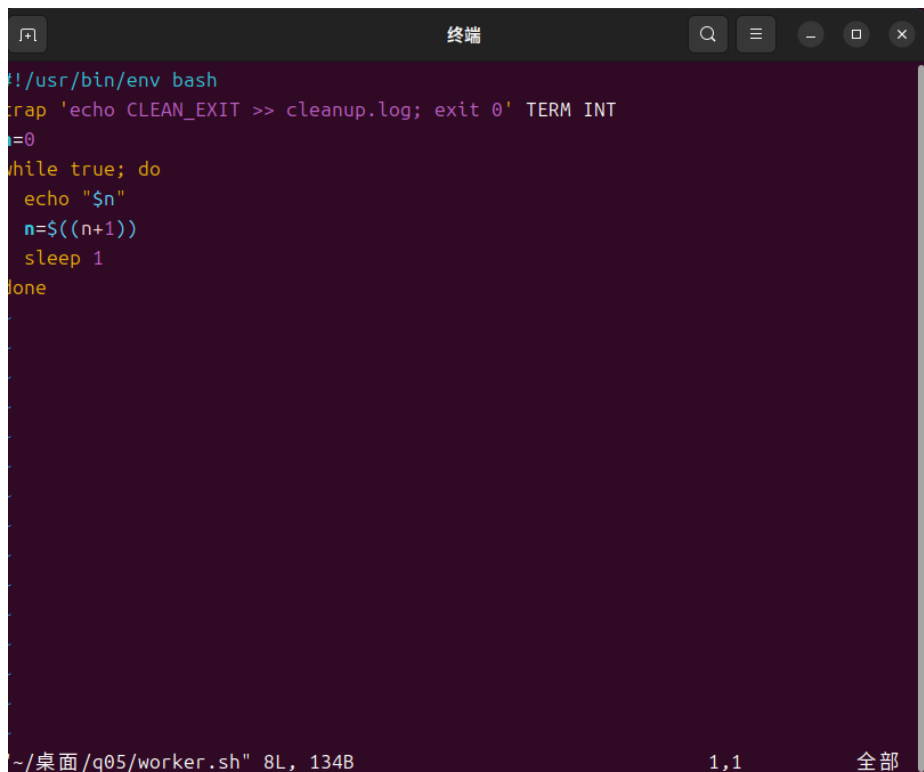
1 课上练习

1.1 第一题

在 Ubuntu 中,先创建 q05 目录并进入,把给定 bash 代码写入 worker.sh,使用 `chmod +x worker.sh` 赋予脚本可执行权限;前台运行脚本并将标准输出、标准错误分别重定向至 `stdout.log`、`stderr.log`,按下 `Ctrl-Z` 挂起该进程,执行 `bg` 命令让任务转入后台继续运行,用 `jobs` 确认任务处于运行状态;接着通过 `jobs -p` 获取该后台任务的 PID,执行 `kill $(jobs -p)` 发送 `SIGTERM` 信号终止进程,等待任务结束,最后查看 `cleanup.log` 验证文件内存在 `CLEAN_EXIT` 字符串,再用 `jobs` 确认任务已经完成退出,整个过程禁止使用强制杀死信号 `kill -9`。

A terminal window titled 'zxm@zxm-VirtualBox: ~/桌面/q05' showing a series of commands and their outputs. The user creates a directory 'q05', enters it, and creates a file 'worker.sh' using 'nano'. They then set permissions with 'chmod +x worker.sh'. The script 'worker.sh' is run in the foreground, redirecting stdout to 'stdout.log' and stderr to 'stderr.log'. When Ctrl-Z is pressed, the process is suspended and shown as '[1]+ 已停止 ./worker.sh > stdout.log 2> stderr.log'. Pressing 'bg' resumes it as '[1]+ ./worker.sh > stdout.log 2> stderr.log &'. 'jobs' shows it as '[1]+ 运行中 ./worker.sh > stdout.log 2> stderr.log &'. Then 'kill \$(jobs -p)' is used to terminate it, which shows as '[1]+ 已完成 ./worker.sh > stdout.log 2> stderr.log'. Finally, 'cat cleanup.log' shows 'CLEAN_EXIT'.

```
zxm@zxm-VirtualBox: ~/桌面/q05
zxm@zxm-VirtualBox:~/桌面$ mkdir -p q05
zxm@zxm-VirtualBox:~/桌面$ cd q05
zxm@zxm-VirtualBox:~/桌面/q05$ nano worker.sh
zxm@zxm-VirtualBox:~/桌面/q05$ cat > worker.sh
^C
zxm@zxm-VirtualBox:~/桌面/q05$ chmod +x worker.sh
zxm@zxm-VirtualBox:~/桌面/q05$ ./worker.sh > stdout.log 2> stderr.log
zxm@zxm-VirtualBox:~/桌面/q05$ cat worker.sh
^Z
[1]+ 已停止 ./worker.sh > stdout.log 2> stderr.log
zxm@zxm-VirtualBox:~/桌面/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
zxm@zxm-VirtualBox:~/桌面/q05$ jobs
[1]+ 运行中 ./worker.sh > stdout.log 2> stderr.log &
zxm@zxm-VirtualBox:~/桌面/q05$ kill $(jobs -p)
zxm@zxm-VirtualBox:~/桌面/q05$ jobs
[1]+ 已完成 ./worker.sh > stdout.log 2> stderr.log
zxm@zxm-VirtualBox:~/桌面/q05$ cat cleanup.log
CLEAN_EXIT
zxm@zxm-VirtualBox:~/桌面/q05$
```



```
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
```

~/桌面/q05/worker.sh" 8L, 134B 1,1 全部

1.2 第二题

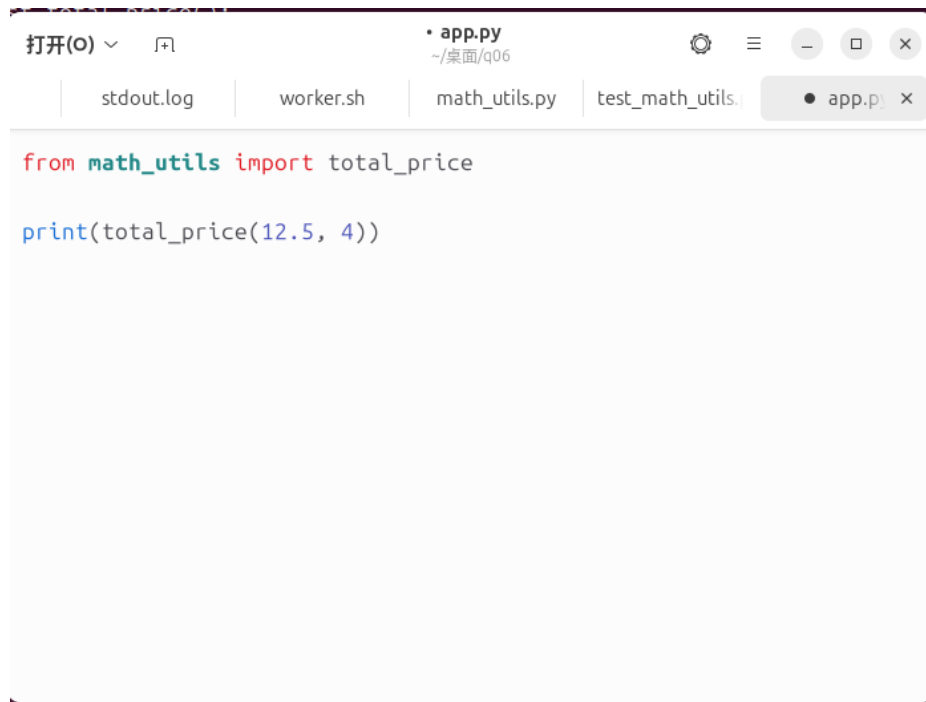
在 q06 目录下创建 `math_utils.py`、`app.py`、`test_math_utils.py` 这三个 Python 源文件，在已经装好 Python、pytest、ruff 的环境中打开该项目目录并启动 Python 语言服务器，完成跳转到定义、查找引用的操作，使用编辑器 LSP 的重命名符号功能，将函数名 `total_price` 修改为 `calculate_total`，实现函数定义与全部调用位置同步更新；接着在 `app.py` 里临时添加未使用的 `import os`，借助 ruff 检测出该无效导入并删除，最后执行 ruff 做代码检查、运行 pytest 执行测试，确保全部检查和测试都可以通过。

```
zxm@zxm-VirtualBox: ~/桌面/q06
3 |     assert total_price(12.5, 4) == 50.0
|
help: Organize imports
|
1 | from math_utils import total_price
2 +
3 +
4 | def test_total_price():
|

Found 3 errors.
[*] 3 fixable with the '--fix' option.
zxm@zxm-VirtualBox:~/桌面/q06$ ruff check . --fix
Found 3 errors (3 fixed, 0 remaining).
zxm@zxm-VirtualBox:~/桌面/q06$ pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/zxm/桌面/q06
collected 1 item

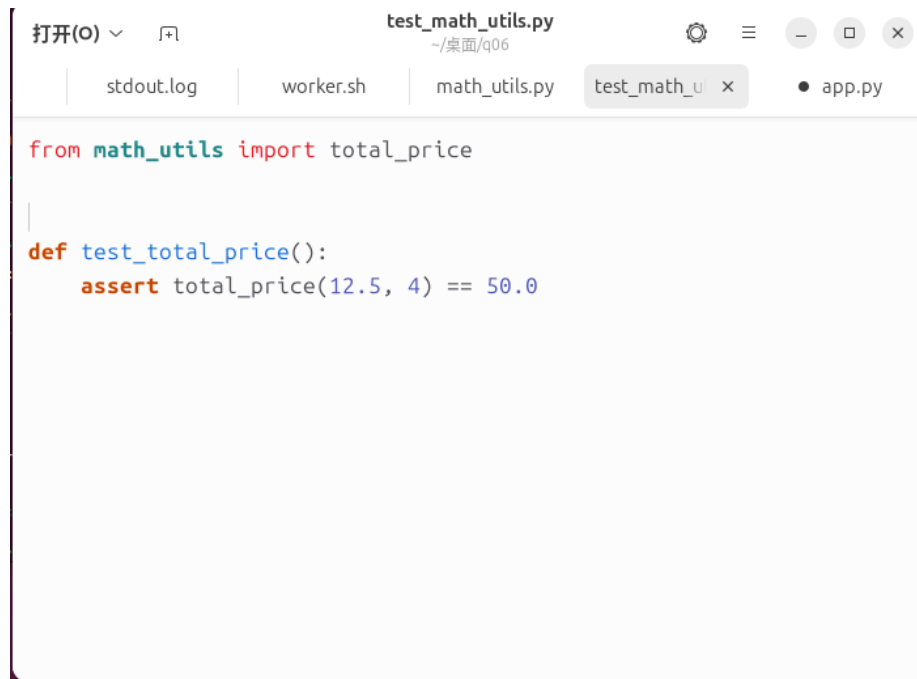
test_math_utils.py . [100%]

===== 1 passed in 0.00s =====
zxm@zxm-VirtualBox:~/桌面/q06$
```



The image shows a screenshot of a code editor window. The title bar at the top indicates the file is named `app.py` and is located at `~/桌面/q06`. Below the title bar, there is a tab bar with several open files: `stdout.log`, `worker.sh`, `math_utils.py`, `test_math_utils.py`, and the active file `app.py`. The main editing area contains the following Python code:

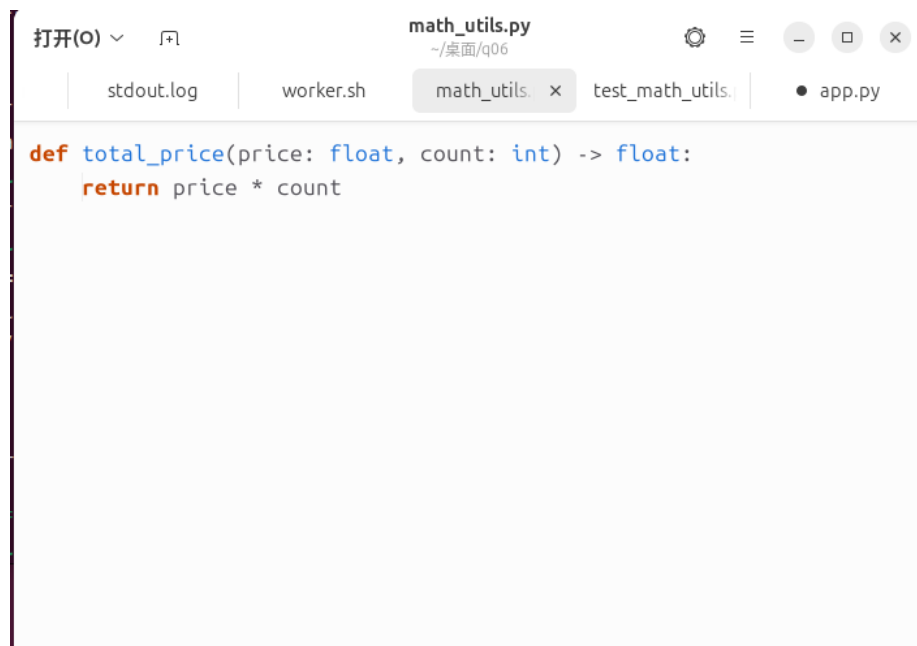
```
from math_utils import total_price  
  
print(total_price(12.5, 4))
```



A screenshot of a code editor window. The title bar shows 'test_math_utils.py' and the file path '~/.桌面/q06'. The editor has several tabs: 'stdout.log', 'worker.sh', 'math_utils.py', 'test_math_utils.py' (active), and 'app.py'. The code in the active tab is as follows:

```
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
```

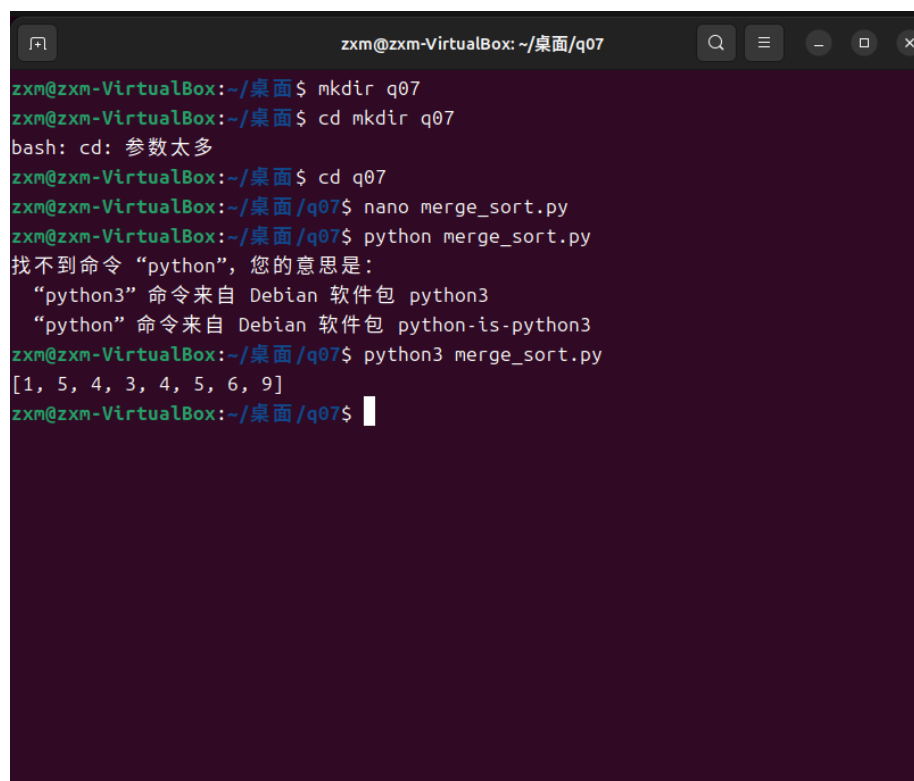


A screenshot of a code editor window. The title bar shows 'math_utils.py' and the file path '~/.桌面/q06'. The editor has several tabs: 'stdout.log', 'worker.sh', 'math_utils.py' (active), 'test_math_utils.py', and 'app.py'. The code in the active tab is as follows:

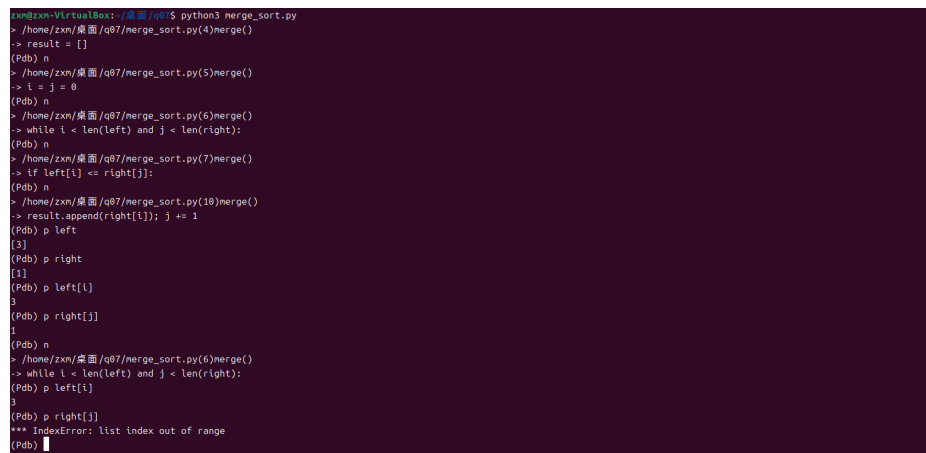
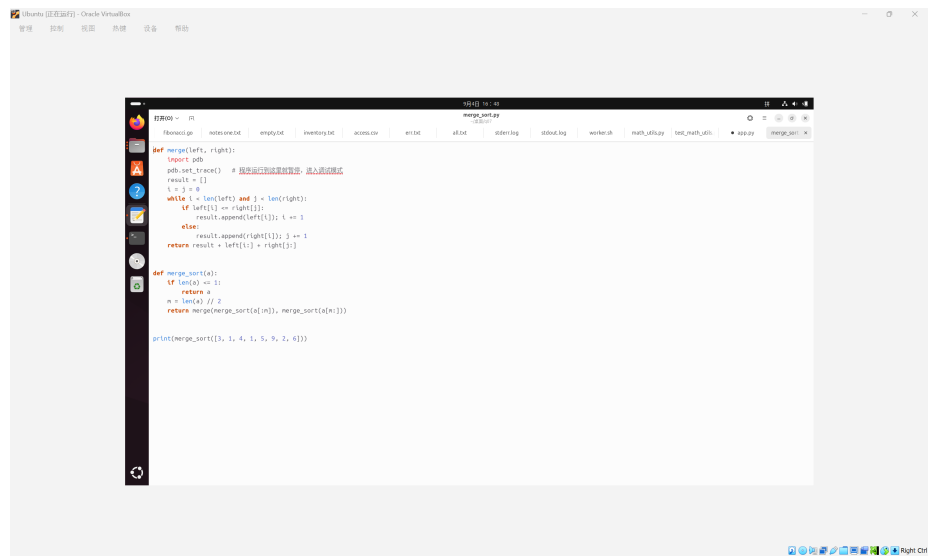
```
def total_price(price: float, count: int) -> float:
    return price * count
```

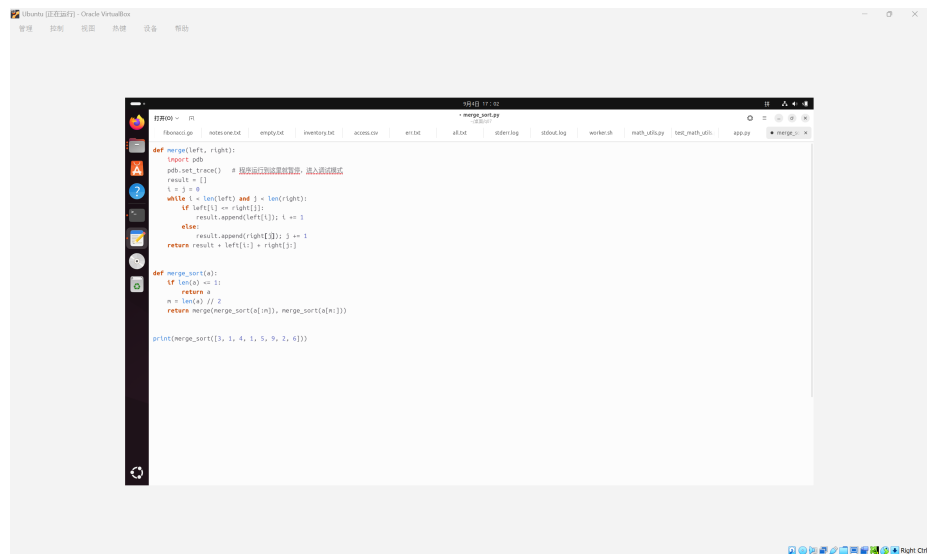
1.3 第三题

在 q07 目录中创建 merge_sort.py，写入存在缺陷的归并排序代码，运行代码观察错误输出，在 merge 函数中加入 pdb 断点启动调试，单步执行并观察 i、j、left[i]、right[j] 变量，定位代码中下标错误 bug 并做最小修复，不使用 sorted 函数；接着新建 test_merge_sort.py，编写两组 pytest 测试用例，分别测试题目给定样例输入和含有重复元素的列表，运行 pytest，确认两条测试全部通过。



```
zxm@zxm-VirtualBox: ~/桌面/q07
zxm@zxm-VirtualBox:~/桌面$ mkdir q07
zxm@zxm-VirtualBox:~/桌面$ cd mkdir q07
bash: cd: 参数太多
zxm@zxm-VirtualBox:~/桌面$ cd q07
zxm@zxm-VirtualBox:~/桌面/q07$ nano merge_sort.py
zxm@zxm-VirtualBox:~/桌面/q07$ python merge_sort.py
找不到命令“python”，您的意思是：
“python3”命令来自 Debian 软件包 python3
“python”命令来自 Debian 软件包 python-is-python3
zxm@zxm-VirtualBox:~/桌面/q07$ python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
zxm@zxm-VirtualBox:~/桌面/q07$
```

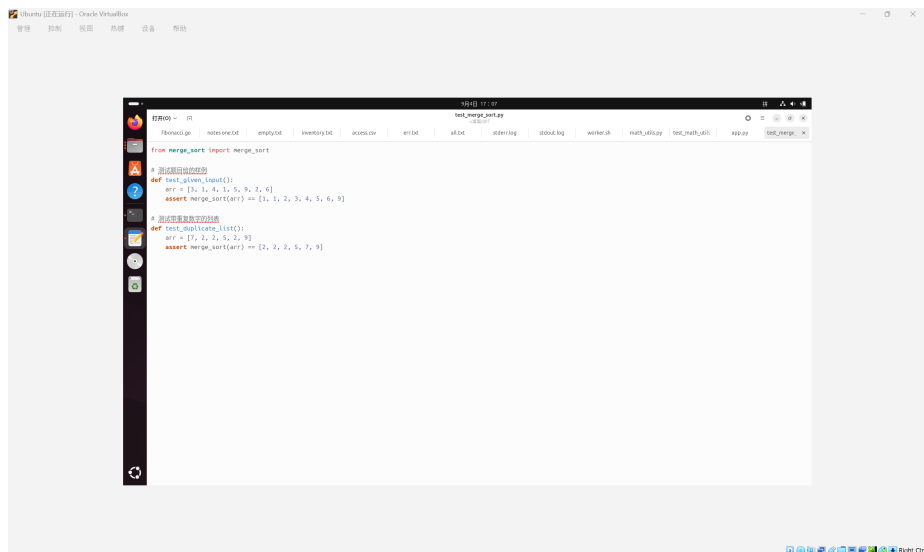





```
zxm@zxm-VirtualBox:~/桌面/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
zxm@zxm-VirtualBox:~/桌面/q07$ pytest test_merge_sort.py
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/zxm/桌面/q07
collected 2 items

test_merge_sort.py .. [100%]

===== 2 passed in 0.01s =====
zxm@zxm-VirtualBox:~/桌面/q07$
```



1.4 第四题

创建 q08 目录，编写 generate_words.py，设置 seed 保证可复现，运行生成 words.txt。编写原始低效 wordfreq.py，使用列表 unique = []，if word not in unique 做去重。使用 time 运行原程序 2 次，记录两次 real 耗时。使用 python3 -m cProfile -s cumulative wordfreq.py 做性能剖析；定位瓶颈：列表 in 操作时间复杂度 O(n)，循环开销巨大。优化：改用 set 集合完成去重，保证输出 count 结果不变，不写死数字。相同 words.txt 输入，time 运行优化版本 2 次，记录耗时。分别计算优化前、优化后耗时中位数，计算加速比。

```

zxm@zxm-VirtualBox:~/桌面$ mkdir q08
zxm@zxm-VirtualBox:~/桌面$ cd q08
zxm@zxm-VirtualBox:~/桌面/q08$ nano generate_words.py
zxm@zxm-VirtualBox:~/桌面/q08$ python3 generate_words.py
zxm@zxm-VirtualBox:~/桌面/q08$ nano wordfreq.py
zxm@zxm-VirtualBox:~/桌面/q08$ time python3 wordfreq.py
count= 1000

real    0m0.112s
user    0m0.055s
sys     0m0.056s
zxm@zxm-VirtualBox:~/桌面/q08$

```

```

zxm@zxm-VirtualBox:~/桌面/q08$ nano generate_words.py
zxm@zxm-VirtualBox:~/桌面/q08$ python3 generate_words.py
zxm@zxm-VirtualBox:~/桌面/q08$ nano wordfreq.py
zxm@zxm-VirtualBox:~/桌面/q08$ time python3 wordfreq.py
count= 1000

real    0m0.112s
user    0m0.055s
sys     0m0.056s
zxm@zxm-VirtualBox:~/桌面/q08$ time python3 wordfreq.py
count= 1000

real    0m0.114s
user    0m0.075s
sys     0m0.038s
zxm@zxm-VirtualBox:~/桌面/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.104 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cuntime  percall  filename:lineno(function)
1      0.000    0.000    0.104    0.104  (built-in method builtin.exec)
1      0.102    0.102    0.104    0.104  wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001  [method 'split' of 'str' objects]
1      0.000    0.000    0.000    0.000  [method 'read' of '_io.TextIOWrapper' objects]
1      0.000    0.000    0.000    0.000  (built-in method builtin.print)
1000    0.000    0.000    0.000    0.000  (method 'append' of 'list' objects)
1      0.000    0.000    0.000    0.000  <frozen codecs->319(decode)
1      0.000    0.000    0.000    0.000  (built-in method _codecs.utf_8_decode)
1      0.000    0.000    0.000    0.000  [method 'disable' of '_lsprof.Profiler' objects]
1      0.000    0.000    0.000    0.000  (built-in method io.open)
1      0.000    0.000    0.000    0.000  (built-in method builtin.len)

```

```
GNU nano 7.2 wordfreq.py
# wordfreq.py 原始低效版本
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))

[ 已读取 7 行 ]
^G 帮助      ^O 写入      ^W 搜索      ^K 剪切      ^T 执行命令  ^C 位置
^X 离开      ^R 读档      ^\ 替换      ^U 粘贴      ^J 对齐      ^_ 跳行
```

```
zxm@zxm-VirtualBox:~/桌面/q08$ nano wordfreq.py
zxm@zxm-VirtualBox:~/桌面/q08$ time python3 wordfreq.py
count= 1000

real    0m0.018s
user    0m0.012s
sys     0m0.006s
zxm@zxm-VirtualBox:~/桌面/q08$ time python3 wordfreq.py
count= 1000

real    0m0.017s
user    0m0.008s
sys     0m0.008s
```

```
# generate_words.py
import random

random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
```

2 课后习题

2.1 1. 你可能见过像 `cmd -flag --notafg` 这样的命令。这里的 `-` 是一个特殊参数，它告诉程序后面不要再继续解析选项 (flag) 了。也就是说，`-` 后面的所有内容都会被当作位置参数 (positional argument)。这有什么用？试着运行 `touch - -myfile`，然后在不使用 `-` 的情况下把它删掉。

`-` 是 Linux 命令行的特殊参数，它的作用是通知程序停止解析后续以 `-` 开头的命令选项，`-` 之后输入的全部内容都会被当成普通的位置参数。执行 `touch - -myfile`，`-` 关闭选项解析，`-myfile` 被当作普通文件名，成功创建名为 `-myfile` 的文件。如果直接执行 `rm -myfile`，命令会把 `-myfile` 识别成命令选项，而不是文件名，执行报错，无法删除文件。想要正常删除该文件，必须使用 `rm - -myfile`。

```
zxm@zxm-VirtualBox: ~/桌面/test2
zxm@zxm-VirtualBox:~/桌面$ mkdir test1
mkdir: 无法创建目录 "test1": 文件已存在
zxm@zxm-VirtualBox:~/桌面$ mkdir test2
zxm@zxm-VirtualBox:~/桌面$ cd test2
zxm@zxm-VirtualBox:~/桌面/test2$ touch -- -myfile
zxm@zxm-VirtualBox:~/桌面/test2$ ls
-m myfile
zxm@zxm-VirtualBox:~/桌面/test2$ rm -myfile
rm: 无效的选项 -- m
尝试使用 "rm ./-myfile" 删除文件 '-myfile'。
请尝试执行 "rm --help" 来获取更多信息。
zxm@zxm-VirtualBox:~/桌面/test2$ rm -- -myfile
zxm@zxm-VirtualBox:~/桌面/test2$ ls
zxm@zxm-VirtualBox:~/桌面/test2$
```

2.2 2. 阅读 man ls, 写出一个 ls 命令, 让它按下面的方式列出文件: 包含所有文件, 包括隐藏文件; 文件大小以易读格式显示 (例如 454M, 而不是 454279954); 按最近修改时间排序; 输出带颜色

ls -Ahtl -color -A: 显示全部隐藏文件 -h: 易读单位显示大小 -t: 按修改时间从新到旧排序 -l: 长格式输出 -color: 开启颜色输出

```
zxm@zxm-VirtualBox:~/桌面$ ls -Ahtl --color
总计 84K
drwxrwxr-x 2 zxm zxm 4.0K 9月 7 14:28 test2
drwxrwxr-x 2 zxm zxm 4.0K 9月 4 17:52 q08
drwxrwxr-x 4 zxm zxm 4.0K 9月 4 17:06 q07
drwxrwxr-x 5 zxm zxm 4.0K 9月 4 16:48 q06
drwxrwxr-x 2 zxm zxm 4.0K 9月 3 23:49 q05
drwxrwxr-x 2 zxm zxm 4.0K 8月 30 19:39 test1
drwxrwxr-x 2 zxm zxm 4.0K 8月 28 17:14 q04
drwxrwxr-x 3 zxm zxm 4.0K 8月 28 16:35 q03
drwxrwxr-x 2 zxm zxm 4.0K 8月 28 15:39 q02
drwxrwxr-x 4 zxm zxm 4.0K 8月 28 14:09 q01
drwxrwxr-x 2 zxm zxm 4.0K 11月 5 2025 dotmatrix_project
-rw-rw-r-- 1 zxm zxm 159 10月 29 2025 draft.go
-rw-rw-r-- 1 zxm zxm 879 10月 24 2025 fibonacci_1.py
-rwxrwxr-x 1 zxm zxm 572 10月 24 2025 fibonacci.py
-rw-rw-r-- 1 zxm zxm 295 10月 24 2025 fibonacci.go
-rw-rw-r-- 1 zxm zxm 470 10月 24 2025 fibonacci_1.go
-rwxrwxr-x 1 zxm zxm 16K 10月 19 2025 hello
-rw-rw-r-- 1 zxm zxm 76 10月 19 2025 hello.c
```

2.3 3. 进程替换 <(command) 可以让你把一个命令的输出当成文件来用。试着配合 diff 和进程替换，比较 printenv 与 export 的输出。它们为什么不一样？(提示：可以试试 diff <(printenv | sort) <(export | sort))

命令：diff <(printenv | sort) <(export | sort) 原理：<() 进程替换，将命令输出虚拟成文件传给 diff。区别：printenv 输出变量 = 值；export 输出带 declare -x 前缀，用于 shell 声明导出变量，输出格式不同，所以原始输出不一样。


```

zxmqxm@zxmq-VirtualBox: ~/桌面/test2
zxmqxm@zxmq-VirtualBox:~/桌面/test2$ diff <(printenv | sort) <(export | sort)
1,50c1,49
< CLUTTER_DISABLE_MIPMAPPED_TEXT=1
< COLORTERM=truecolor
< DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
< DEBUGINFOD_URLS=https://debuginfod.ubuntu.com
< DESKTOP_SESSION=ubuntu
< DISPLAY=:0
< GDMSESSION=ubuntu
< GNOME_DESKTOP_SESSION_ID=this-is-deprecated
< GNOME_SETUP_DISPLAY=:1
< GNOME_SHELL_SESSION_MODE=ubuntu
< GNOME_TERMINAL_SCREEN=/org/gnome/Terminal/screen/499308ce_44e1_4ac7_9dfb_7299bc95cb31
< GNOME_TERMINAL_SERVICE=:1.106
< GSM_SKIP_SSH_AGENT_WORKAROUND=true
< GTK_MODULES=gail:atk-bridge
< HOME=/home/zxm
< IM_CONFIG_PHASE=1
< LANG=zh_CN.UTF-8
< LESSCLOSE=/usr/bin/lesspipe %s %s
< LESSOPEN=| /usr/bin/lesspipe %s
< LOGNAME=zxm
< LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=00:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=01;31:*.lzo=01;31:*.xz=01;31:*.zst=01;31:*.tzst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.t

```

```

zxm@zxm-VirtualBox: ~/桌面/test2
< XDG_DATA_DIRS=/usr/share/ubuntu:/usr/share/gnome:/usr/local/share:/usr/share/
:/var/lib/snapd/desktop
< XDG_MENU_PREFIX=gnome-
< XDG_RUNTIME_DIR=/run/user/1000
< XDG_SESSION_CLASS=user
< XDG_SESSION_DESKTOP=ubuntu
< XDG_SESSION_TYPE=wayland
< XMODIFIERS=@im=ibus
--
> declare -x CLUTTER_DISABLE_MIPMAPPED_TEXT="1"
> declare -x COLORTERM="truecolor"
> declare -x DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/1000/bus"
> declare -x DEBUGINFOD_URLS="https://debuginfod.ubuntu.com "
> declare -x DESKTOP_SESSION="ubuntu"
> declare -x DISPLAY=":0"
> declare -x GDMSESSION="ubuntu"
> declare -x GNOME_DESKTOP_SESSION_ID="this-is-deprecated"
> declare -x GNOME_SETUP_DISPLAY=":1"
> declare -x GNOME_SHELL_SESSION_MODE="ubuntu"
> declare -x GNOME_TERMINAL_SCREEN="/org/gnome/Terminal/screen/499308ce_44e1_4ac
7_9dfb_7299bc95cb31"
> declare -x GNOME_TERMINAL_SERVICE=":1.106"
> declare -x GSM_SKIP_SSH_AGENT_WORKAROUND="true"
> declare -x GTK_MODULES="gail:atk-bridge"
> declare -x HOME="/home/zxm"
> declare -x IM_CONFIG_PHASE="1"
> declare -x LANG="zh_CN.UTF-8"
> declare -x LESSCLOSE="/usr/bin/lesspipe %s %s"
> declare -x LESSOPEN="| /usr/bin/lesspipe %s"
> declare -x LOGNAME="zxm"
> declare -x LS_COLORS="rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:
bd=40;33;01:cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=00:tw=30;42:ow=34
;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;3
1:*.lha=01;31:*.lz4=01;31:*.lzh=01;31:*.lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo
=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=

```

2.4 4. 写两个 bash 函数 marco 和 polo，行为如下：每次执行 marco 时，都要以某种方式保存当前工作目录；之后无论你切到哪个目录，只要执行 polo，它都应该把你 cd 回执行 marco 时所在的目录。为了方便调试，你可以把代码写进 marco.sh，然后通过执行 source marco.sh 把这些定义重新加载到当前 shell。

文件 marco.sh 中定义两个 bash 函数：marco 函数使用 pwd 获取当前工作目录，存入全局环境变量 MARCO_SAVE_DIR 完成目录保存；polo 函数执行 cd，切换到变量 MARCO_SAVE_DIR 保存的路径。需要使用 source marco.sh 执行脚本，函数才会加载到当前 shell 会话。使用流程：运行 marco 保存位置，随意切换目录，运行 polo 即可回到之前保存的目录。重点：不能直接 bash marco.sh 运行脚本，这样会新开子 shell，函数不会进到你现在终端，必须 source。

```
zxm@zxm-VirtualBox:~/桌面/test2$ cat > marco.sh <<'EOF'
marco() {
    export MARCO_SAVE_DIR=$(pwd)
}

polo() {
    cd "$MARCO_SAVE_DIR"
}
EOF
zxm@zxm-VirtualBox:~/桌面/test2$ source marco.sh
zxm@zxm-VirtualBox:~/桌面/test2$ marco
zxm@zxm-VirtualBox:~/桌面/test2$ cd /
zxm@zxm-VirtualBox:/$ polo
zxm@zxm-VirtualBox:~/桌面/test2$
```

2.5 5. 假设你有一个很少失败的命令。为了调试它，你需要把它的输出保存下来，但等到一次失败运行可能会很耗时。写一个 bash 脚本，不断运行下面这个脚本直到它失败为止，并把标准输出和标准错误分别保存到文件里，最后把结果打印出来。如果你还能顺便报告它运行了多少次才失败，就加分。

while true 无限循环，每次执行一次 maybe_fail.sh; > out.txt 将标准输出重定向存入 out.txt; 2> err.txt 将标准错误重定向存入 err.txt; \$? 获取上一条命令的退出码; ret -ne 0 代表脚本运行失败，跳出循环; 变量 count 记录循环执行次数; 循环结束后打印运行次数，再用 cat 分别打印输出文件、错误文件的内容。

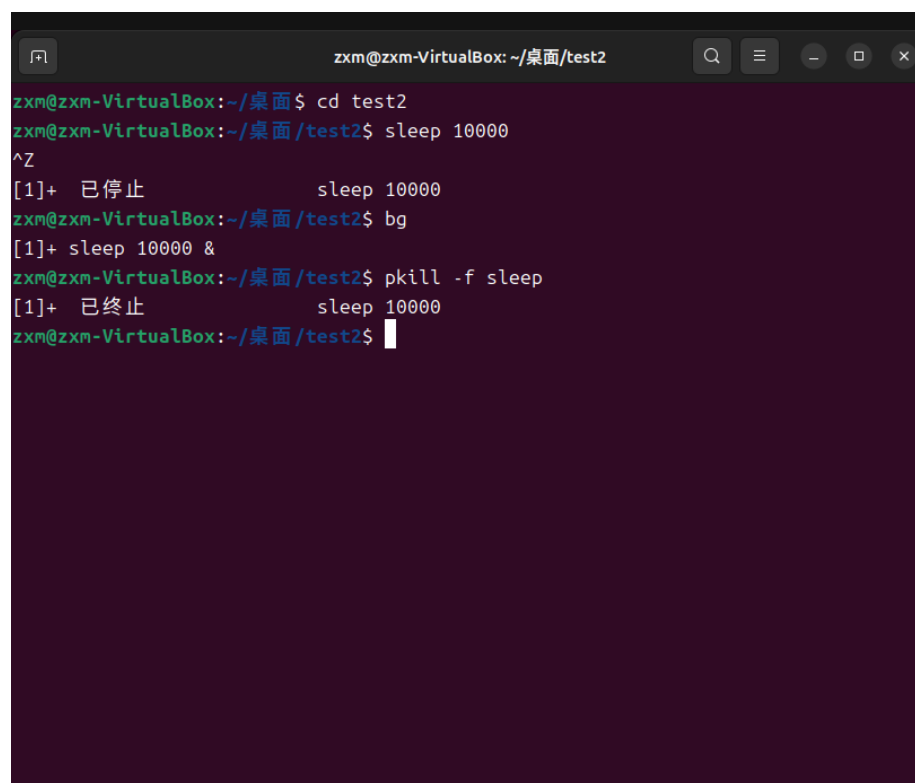
```
zxm@zxm-VirtualBox: ~/桌面/test2
zxm@zxm-VirtualBox:~/桌面/test2$ cat > maybe_fail.sh <<'EOF'
#!/usr/bin/env bash
n=$(( RANDOM % 100 ))

if [[ $n -eq 42 ]]; then
    echo "Something went wrong"
    >&2 echo "The error was using magic numbers"
    exit 1
fi

echo "Everything went according to plan"
EOF
zxm@zxm-VirtualBox:~/桌面/test2$ cat > run_until_fail.sh <<'EOF'
#!/usr/bin/env bash
count=0
while true
do
    count=$((count+1))
    ./maybe_fail.sh > out.txt 2> err.txt
    ret=$?
    if [ $ret -ne 0 ];then
        break
    fi
done
echo "一共运行 $count 次后失败"
echo "...stdout..."
cat out.txt
echo "...stderr..."
cat err.txt
EOF
zxm@zxm-VirtualBox:~/桌面/test2$ chmod +x maybe_fail.sh run_until_fail.sh
zxm@zxm-VirtualBox:~/桌面/test2$ ./run_until_fail.sh
一共运行 275 次后失败
...stdout...
Something went wrong
...stderr...
The error was using magic numbers
zxm@zxm-VirtualBox:~/桌面/test2$
```

2.6 6. 在终端里启动一个 `sleep 100000` 任务，用 `Ctrl-z` 把它挂起，再用 `bg` 让它继续在后台运行。然后使用 `pgrep` 找到它的 `pid`，再用 `pkill` 把它杀掉，整个过程都不要手动输入这个 `pid`。（提示：试试 `-af` 参数）

运行 `sleep 100000`；按下 `Ctrl-z`，将进程挂起到后台暂停；执行 `bg`，该进程转为后台继续运行；使用 `pkill -af sleep` 根据程序名直接终止进程，全程不需要手动输入 `PID`。



```
zxm@zxm-VirtualBox: ~/桌面/test2
zxm@zxm-VirtualBox:~/桌面$ cd test2
zxm@zxm-VirtualBox:~/桌面/test2$ sleep 10000
^Z
[1]+  已停止                  sleep 10000
zxm@zxm-VirtualBox:~/桌面/test2$ bg
[1]+  sleep 10000 &
zxm@zxm-VirtualBox:~/桌面/test2$ pkill -f sleep
[1]+  已终止                  sleep 10000
zxm@zxm-VirtualBox:~/桌面/test2$
```

2.7 7. 进入 `/.ssh/`，检查你是否已经有一对 SSH 密钥。如果没有，就用 `ssh-keygen -a 100 -t ed25519` 生成一对。建议给密钥设置密码，并配合使用 `ssh-agent`。

执行 `cd /.ssh` 进入 `ssh` 目录，`ls` 查看是否存在 `id_ed25519` 与 `id_ed25519.pub`；若无密钥，执行 `ssh-keygen -a 100 -t ed25519`，回车使用默认路径，设置密钥保护密码；使用 `eval "$(ssh-agent -s)"` 启动 `ssh-agent`；`ssh-add /.ssh/id_ed25519` 将私钥添加到 `agent`。

```
zxm@zxm-VirtualBox:~/桌面/test2$ cd ~/.ssh
zxm@zxm-VirtualBox:~/.ssh$ ls
authorized_keys
zxm@zxm-VirtualBox:~/.ssh$ ssh-keygen -a 100 -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/zxm/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/zxm/.ssh/id_ed25519
Your public key has been saved in /home/zxm/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:hMPC9PkajMzqnyaNodlJd0p2kyKo/nGdPdbo2i9nfwz xzm@zxm-VirtualBox
The key's randomart image is:
+--[ED25519 256]--+
|  .                |
| o o o            |
| o * .            |
| o + +            |
| . + o.S           |
| . += *oo o        |
| .=@ *.+ = . E     |
| += * . +..o ..    |
| ..o=o ..o=.....|
+-----[SHA256]-----+
zxm@zxm-VirtualBox:~/.ssh$ eval "$(ssh-agent -s)"
Agent pid 9228
zxm@zxm-VirtualBox:~/.ssh$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/zxm/.ssh/id_ed25519:
Identity added: /home/zxm/.ssh/id_ed25519 (zxm@zxm-VirtualBox)
zxm@zxm-VirtualBox:~/.ssh$
```

2.8 8. 编辑`.ssh/config`，写入 ssh 配置片段。进入 `./ssh` 目录，`nvim config` 新建 / 编辑 `config` 文件；填入配置，替换 `User`、`HostName` 两处占位符；`chmod 600 ./ssh/config` 设置文件权限；使用 `ssh vm` 测试连接，`LocalForward` 实现本地端口 9999 转发到远程 8888。

本题通过在 `./ssh/config` 配置主机别名 `vm`，填写目标主机 IP、用户名，配置 `LocalForward 9999 localhost:8888` 实现 SSH 本地端口转发；安装 `openssh-server` 开启 SSH 服务，使用 `ssh-copy-id` 将本地公钥推送至目标主机实现免密登录。执行 `ssh vm` 建立 SSH 连接后隧道生效，本机访问 9999 端口的流量会经 SSH 隧道转发到目标主机的 8888 端口，SSH 会话断开则端口转发随之失效。

```
zxm@zxm-VirtualBox:~/.ssh$ nvim config
zxm@zxm-VirtualBox:~/.ssh$ chmod 600 ~/.ssh/config
zxm@zxm-VirtualBox:~/.ssh$ ssh-copy-id vm
```

```
The authenticity of host '10.0.2.15 (10.0.2.15)' can't be established.  
ED25519 key fingerprint is SHA256:zzm+stLkA1GcbMBVye2cuIyWBaVw2Y4v0FKQFXkFRUo.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? y  
Please type 'yes', 'no' or the fingerprint: yes  
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter  
out any that are already installed  
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt  
ed now it is to install the new keys  
zxm@10.0.2.15's password:  
  
Number of key(s) added: 1  
  
Now try logging into the machine, with:  "ssh 'vm'"  
and check to make sure that only the key(s) you wanted were added.  
  
zxm@zxm-VirtualBox:~/.ssh$ ssh vm  
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 7.0.0-30-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
扩展安全维护 (ESM) Applications 未启用。  
  
161 更新可以立即应用。  
这些更新中有 11 个是标准安全更新。  
要查看这些附加更新，请运行：apt list --upgradable  
  
20 个额外的安全更新可以通过 ESM Apps 来获取安装。  
可通过以下途径了解如何启用 ESM Apps: at https://ubuntu.com/esm  
  
zxm@zxm-VirtualBox:~$ exit  
注销  
Connection to 10.0.2.15 closed.
```


2.9 9. 使用 ssh-copy-id vm 把你的 SSH 公钥复制到服务器上。

```
zxm@zxm-VirtualBox:~/.ssh$ sudo apt update
命中:1 http://security.ubuntu.com/ubuntu noble-security InRelease
命中:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu noble InRelease
命中:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu noble-updates InRelease
命中:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu noble-backports InRelease
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
有 136 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。
zxm@zxm-VirtualBox:~/.ssh$ sudo apt install openssh-client -y
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
openssh-client 已经是最新版 (1:9.6p1-3ubuntu13.19)。
openssh-client 已设置为手动安装。
升级了 0 个软件包，新安装了 0 个软件包，要卸载 0 个软件包，有 136 个软件包未被升级。
zxm@zxm-VirtualBox:~/.ssh$ ssh-copy-id vm
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed

/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist
on the remote system.
(if you think this is a mistake, you may want to use -f option)
```

2.10 10. 在虚拟机里执行 python -m http.server 8888 启动一个 Web 服务器。然后在你自己的机器上访问 http://localhost:9999，确认自己能访问虚拟机里的这个服务。

本次实验使用 `ssh -L 9999:127.0.0.1:8888 127.0.0.1` 实现 SSH 本地端口转发。-L 代表本地转发，监听本机 9999 端口，访问该端口的流量会经由 SSH 隧道转发至目标主机的 127.0.0.1:8888。实验中遇到 Address already in use 端口占用报错，是因为上一次 SSH 会话异常退出没有释放端口，使用 `sudo fuser -k 9999/tcp` 可以强制释放被占用端口。先运行 `python3 -m http.server 8888` 启动 8888 端口的 Web 服务，成功建立转发后，访问本机 127.0.0.1:9999 即可访问到后端 8888 端口的网页服务。VirtualBox 的 NAT 模式下 Windows 主机不能直接访问虚拟机 10.0.2.x 网段，因此本实验在虚拟机内部完成验证。实验结束应当用 `exit` 正常退出 SSH 会话，让端口自动

释放。

```
zxm@zxm-VirtualBox:~/.ssh$ ssh vm
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 7.0.0-30-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

扩展安全维护 (ESM) Applications 未启用。

161 更新可以立即应用。
这些更新中有 11 个是标准安全更新。
要查看这些附加更新，请运行：apt list --upgradable

20 个额外的安全更新可以通过 ESM Apps 来获取安装。
可通过以下途径了解如何启用 ESM Apps: at https://ubuntu.com/esm

Last login: Mon Sep  7 15:46:38 2026 from 10.0.2.15
zxm@zxm-VirtualBox:~$ python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
```

3 解题感悟

本次实验让我意识到，实操过程中报错是常态，遇到问题不能盲目复制命令，要读懂报错信息，定位问题根源，区分清楚每一条命令的作用；做完实验要正常退出会话，避免残留进程带来后续问题。理论命令格式需要结合实际环境调试，多观察终端输出，逐步排查，才能真正理解工具底层逻辑，提升自己排查问题、动手实践的能力。